

A Fast and Memory Efficient MLCS Algorithm by Character Merging for DNA Sequences Alignment

Sen Liu^a, Yuping Wang^{a,*}, Wuning Tong^a and Shiwei Wei^a

^aSchool of Computer Science and Technology, Xidian University, Xi'an, 710071, China.

*To whom correspondence should be addressed.

Abstract

Motivation: Multiple Longest Common Subsequence (MLCS) problem is searching all longest common subsequences of multiple character sequences. It appears in many fields such as data mining, DNA alignment, bioinformatics, text editing and so on. With the increasing of sequence length and number of sequences, the existing dynamic programming algorithms and the dominant point based algorithms become ineffective and inefficient, especially for large-scale MLCS problems.

Results: In this paper, by considering the characteristics of DNA sequences with many consecutively repeated characters, we first design a character merging scheme which merges the consecutively repeated characters in the sequences. As a result, it shortens the length of sequences considered and saves the space of storing all sequences. To further reduce the space and time costs, we construct a Weighted Directed Acyclic Graph (WDAG) which is much smaller than widely used DAG for MLCS problems. Based on these techniques, we propose a fast and memory efficient algorithm for MLCS problems. Finally, the experiments are conducted and the proposed algorithm is compared with several state-of-the-art algorithms. The experimental results show that the proposed algorithm performs better than the compared state-of-the-art algorithms in both time and space costs.

Contact: ywang@xidian.edu.cn

1 Introduction

Multiple Longest Common Subsequence (MLCS) problem is a classic but challenging problem which has wide applications in many fields such as data mining, DNA alignment, bioinformatics, text editing and so on. Especially, finding MLCS for DNA sequences is a very effective method for DNA sequence similarity analysis in bioinformatics (Andrade *et al.* (2014), Kim and Shanklin (1998), Chaabane (2018), Umar *et al.* (2015) and Podlipnĭĭ (2009)). With the fast development of gene expression and DNA technology, the number and length of DNA sequences increase greatly. This requires MLCS algorithms be more efficient. DNA sequences consist of characters $\Sigma = \{A, C, G, T\}$, and finding MLCS for multiple DNA sequences has been proven to be an NP-hard problem (Maier (1978)). As the length and number of DNA sequences increase, the time and space costs of solving this problem grow exponentially. Therefore, it is necessary to design more efficient algorithm. Finding the longest common subsequence problem of two sequences is called the LCS problem. Many algorithms (Sankoff (1972), Hirschberg (1977), Smith *et al.* (1981), Masek and Paterson (1980), Guerra (1992) and Hsu and Du (1984)) have been proposed for this problem. The widely used one is the dynamic programming algorithm, and the time complexity of the traditional dynamic programming algorithm is $O(n^2)$ (Guerra (1992) where n is the length of the sequences. As the number of sequences increases, the space and time complexity of the dynamic programming

algorithm increases exponentially and thus the dynamic programming algorithm cannot efficiently handle MLCS problems, especially the problems with a lot of sequences. To efficiently solve MLCS problems, in 1977, Hunt proposed an algorithm called the dominant point based algorithm (Hunt and Szymanski (1977)). This method enumerates all possible common subsequences by constructing a Directed Acyclic Graph (DAG) in which each path from the source node of DAG to any node is a common subsequence and each longest path corresponds a longest common subsequence. Thus, this algorithm transforms the MLCS problem into the longest path problem from the source node to the destination node. The dominant point based algorithm has been shown to be much more effective and efficient than the dynamic programming algorithm for MLCS problems (Wang *et al.* (2011)). It greatly reduces the time and space complexity of the dynamic programming algorithm, but when the number of sequences increases, the space and time complexity still increases exponentially. To enhance the performance of the dominant point based algorithm, a series of improved dominant point based algorithms were proposed, such as (Wang *et al.* (2011), Korkin (2001), Chen *et al.* (2006), Yang *et al.* (2010), Yang *et al.* (2013), Li *et al.* (2016), Peng and Wang (2017), Aravanis *et al.* (2017) and Gustavsson and Syberfeldt (2018)), etc. The algorithm of Korkin (Korkin (2001)) is a parallel version of the original dominant point based algorithm. Chen proposed a new algorithm called FAST_LCS (Chen *et al.* (2006)) by using a data structure called the successor table. The successor table can quickly create subsequent points and remove dominated points in DAG, and thus Fast_LCS greatly reduces

the time required to construct DAG. Based on FAST_LCS, Wang proposed an algorithm called Quick-DPPAR (Wang *et al.* (2011)) which improves the process of removing the dominated points of DAG. Yang (Yang *et al.* (2010)) proposed a parallel MLCS algorithm based on cloud computing platform, but the synchronization cost is too high when solving large-scale MLCS problems. In 2016, Li and Wang proposed a new algorithm called Top_MLCS (Li *et al.* (2016)) which deleted all redundant points (all repeated and all dominated points) by designing an Irredundant Common Subsequence Graph (ICSG). ICSG is much smaller than DAG and thus Top_MLCS is much efficient than the above mentioned algorithms for large-scale MLCS problems. However, when the scale of the MLCS problems is extended to a certain extent, Top_MLCS requires a lot of time and space to create and store ICSG, and cannot solve the MLCS problems in an acceptable time and space costs. In order to solve large-scale DNA sequences alignment more effectively, we propose a new fast and memory efficient MLCS algorithm in this paper. By analyzing the characteristics of DNA sequences, we find that there are a large number of consecutively repeated characters in DNA sequences. To use this characteristic, we design a scheme called character merging to remove the repeated characters and reduce the space and time complexity. Based on this scheme, we propose an efficient algorithm. Our main contributions are as follows:

- **Propose a character merging scheme:** For each sequence given, merging scheme will merge consecutively repeated characters and then use a vector to record the number of the consecutively repeated characters for each merged sequence. In this way, the length of the sequence will be shortened and the cost of searches for successor points will be reduced.
- **Design a Weighted Directed Acyclic Graph (WDAG):** For the merged sequences (shortened sequences), we first construct a Directed Acyclic Graph (DAG) using the method similar to that in (Li *et al.* (2016)). However, this DAG is much smaller than that constructed in (Li *et al.* (2016)) for the same sequences. Then, we define the edge weight by the number of the repeated character at the end node of DAG. In this way, we construct a Weighted Directed Acyclic Graph (WDAG) which is much smaller than that constructed in (Li *et al.* (2016)).
- **Propose an efficient method to find MLCS by searching the longest paths of WDAG:** Each longest path of WDAG from the source point to the ending point is an MLCS. We propose an efficient method to look for the longest paths of WDAG.
- **The proposed algorithm can solve much larger scale problems than the existing algorithms, and is much faster with lower space and time costs:** Because DNA sequences have a lot of consecutively repeated characters, the shortened sequences (by merging the consecutively repeated characters) become much shorter. Thus the proposed algorithm can handle much large scale DNA sequences than the existing algorithms can, and uses much less time and space than the existing algorithms do.

The rest of this article is organized as follows. Section II briefly describes the LCS/MLCS problem and reviews the related algorithms, mainly the dynamic programming algorithm and dominant point based algorithms. Section III introduces the proposed algorithm for solving the LCS/MLCS problem by using characters merging. Section IV analyzes the performance of the proposed algorithm and compares it with other state-of-the-art algorithms by experiments. Finally, Section V concludes this paper.

2 Related work

2.1 Definitions of LCS/MLCS Problems

In order to better understand and solve the LCS/MLCS problems, we briefly introduce the following definitions (Smith *et al.* (1981), Guerra (1992), Cormen *et al.* (2009)):

Definition 1 : Give a sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, another sequence $Y = \langle y_1, y_2, \dots, y_m \rangle$ is a subsequence of X when the following conditions are met. That is, there is a strictly increasing subscript sequence $\langle i_1, i_2, \dots, i_m \rangle$ of X , which satisfies $x_{i_j} = y_j$ for all $j = 1, 2, \dots, m$. For example, $Y = ATAG$ is a subsequence of $X = ACTGAGC$, and the corresponding subscript sequence is $\langle 1, 3, 5, 6 \rangle$.

Definition 2 : Give two sequences X and Y , if Z is a subsequence of both X and Y , then we call Z is a common subsequence of sequences X and Y . For example, for sequences $X = ACTGTACG$ and $Y = CATGAACG$, the sequence $ATGA$ is a common subsequence of X and Y .

Definition 3 : In the common subsequence set of sequences X and Y , any longest sequence is called a longest common subsequence of X and Y , and the longest common subsequence may be not unique and there may be multiple longest common subsequences.

Definition 4 : The Longest Common Subsequences (LCS) problem is to find all longest common subsequences of given sequences $X = \langle x_1, x_2, \dots, x_n \rangle$ and $Y = \langle y_1, y_2, \dots, y_m \rangle$. Multiple Longest Common Subsequences (MLCS) problem is to find all longest common subsequences of three or more given sequences in a given set S ($\|S\| \geq 3$) of sequences.

Definition 5 : Give a sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, for $i = 0, 1, \dots, n$, define $X_i = \langle x_1, x_2, \dots, x_i \rangle$ as the i -th prefix of X . For example, if $X = ACTGTACA$, then $X_5 = ACTGT$ is the 5-th prefix of X , and X_0 is an empty sequence.

The dynamic programming algorithm is a widely used algorithm for solving MLCS problems with the following well known optimal substructure property (Cormen *et al.* (2009)):

Theorem 1 (optimal substructure property): Let $X = \langle x_1, x_2, \dots, x_n \rangle$ and $Y = \langle y_1, y_2, \dots, y_m \rangle$ are two given sequences, $Z = \langle z_1, z_2, \dots, z_k \rangle$ is a LCS of X and Y .

1. If $x_n = y_m$, then $z_k = x_n = y_m$, and z_{k-1} is a LCS of X_{n-1} and Y_{m-1} .
2. If $x_n \neq y_m$, then $z_k \neq x_n$ means that Z is a LCS of X_{n-1} and Y .
3. If $x_n \neq y_m$, then $z_k \neq y_m$ means Z is a LCS of X and Y_{m-1} .

In the following, we use Σ to denote the character set forming the sequences and use S to denote the set of the given sequences. Also, we use $LCS(S)$ to represent set of all longest common subsequences of the sequences in set S . For example, for sequences $X = ACTAGTGC$ and $Y = TGCTAGCA$ consisting of four characters A, C, G, T , the character set $\Sigma = \{A, C, G, T\}$ and the sequence set $S = \{X, Y\}$. The longest common subsequences of X and Y are $CTGA$ and $ATGA$. Thus, $LCS(S) = \{TGTGC, CTAGC\}$.

2.2 Dynamic Programming Algorithms

From Theorem 1, we can see the LCS problem has the good substructure properties. When solving a LCS of $X = \langle x_1, x_2, \dots, x_n \rangle$ and $Y = \langle y_1, y_2, \dots, y_m \rangle$, we need to solve one or two subproblems. If $x_n = y_m$, we should solve one LCS problem of X_{n-1} and Y_{m-1} first, and then we can append $x_n = y_m$ to the end of this LCS to get a LCS of X and Y . If $x_n \neq y_m$, we have to solve two subproblems: the LCS problem of X_{n-1} and Y and the LCS problem of X and Y_{m-1} . The longer one of these two LCS is a LCS of X and Y (Cormen *et al.* (2009)). Since these cases

cover all possibilities, in principle, the LCS of X and Y can be obtained by recursively solving the subproblems.

The detail of this recursive process of the dynamic programming algorithm for LCS problems is as follows: We use score matrix $L[i, j]$ to represent the LCS's length of X_i and Y_j . If $i = 0$ or $j = 0$, that is, when the sequences are empty, the length of the LCS is set to 0. According to Theorem 1, we can get the following recursive **formula(1)**(Sankoff (1972)):

$$L[i, j] = \begin{cases} 0 & \text{if } i \text{ or } j = 0 \\ L[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(L[i, j-1], L[i-1, j]) & \text{if } x_i \neq y_j \end{cases} \quad (1)$$

According to formula (1), we can recursively calculate the LCS of two sequences from bottom to up. The dynamic programming LCS algorithm uses two sequences $X = \langle x_1, x_2, \dots, x_n \rangle$ and $Y = \langle y_1, y_2, \dots, y_m \rangle$ as input, stores the values of $L[i, j]$ in the table $L[0, 1, \dots, n, 0, 1, \dots, m]$, and calculates the entries in row-major order. The algorithm also maintains a table $c[1, 2, \dots, n, 1, 2, \dots, m]$ to help construct the optimal solution. The entry $L[i, j]$ corresponds to the optimal solution of the subproblem (Masek and Paterson (1980)).

For example, given the sequences $X = ATTTGCGGA$ and $Y = TTATGCCA$, by formula (1), we can find that LCS of sequences X and Y is $LCS(X, Y) = \{TTTCA, TTTGA\}$.

Similarly, for a set $S = \{S_1, S_2, \dots, S_d\}$ of $d(d \geq 3)$ sequences, we define a score matrix $L[i_1, i_2, \dots, i_d]$ which represents the MLCS of the i_j -th prefix of sequence S_j for $j = 1, 2, \dots, d$, respectively. We can get a similar recursive formula as follows (Li et al. (2016)):

$$L[i_1, i_2, \dots, i_d] =$$

$$\begin{cases} 0 & \exists i_j = 0, 1 \leq i \leq d \\ L[i_1-1, i_2-1, \dots, i_d-1] + 1 & \text{if } S_1[i_1] = \dots = S_d[i_d] \\ \max(\bar{L}) & \text{otherwise} \end{cases} \quad (2)$$

where $\bar{L} = \{L[i_1-1, i_2, \dots, i_d], L[i_1, i_2-1, \dots, i_d], \dots, L[i_1, i_2, \dots, i_d-1]\}$

Note that time complexity and space complexity of the dynamic programming algorithm for the LCS/MLCS problem with d sequences and length n are $O(n^d)$ (Guerra (1992)). Thus, the time and space consumption of the dynamic programming algorithm is unacceptable when d and n are large.

2.3 Dominant Point based Algorithms

The dominant point based algorithms are currently the mainstream algorithms for solving LCS/MLCS problems, and many efficient algorithms are constructed based on the idea of this kind of algorithms. The construction of DAG in dominant point based algorithms is the key for solving the LCS/MLCS problems. To better understand the principle of the dominant point based algorithms, we first introduce the following definitions (Korkin (2001), Gustavsson and Syberfeldt (2018)):

Definition 6 : Given the character set Σ , the sequences set containing d sequences is denoted by $T = \{S_1, S_2, \dots, S_i, \dots, S_d\}$, ($1 \leq i \leq d$), where $|S_i| = n$ and $S_i[p_i] \in \Sigma$ represents the p_i -th character of the i -th sequence S_i . For d sequences, if $p_1, p_2, \dots, p_d$ satisfy $S_1[p_1] = S_2[p_2] = \dots = S_d[p_d] = c \in \Sigma$, then d dimensional point $p = (p_1, p_2, \dots, p_d)$ is called a matched point of the sequence set $T = \{S_1, S_2, \dots, S_i, \dots, S_d\}$, and its corresponding character is character c .

Definition 7 : For two matched points $p = (p_1, p_2, \dots, p_d)$ and $q = (q_1, q_2, \dots, q_d)$ for a given sequences set $T = \{S_1, S_2, \dots, S_i, \dots, S_d\}$, if $p_i = q_i$ for $i = 1, 2, \dots, d$, we say $p = q$. If $p_i \leq q_i$ for $i = 1, 2, \dots, d$ and $p_j < q_j$ for some $1 \leq j \leq d$, we say p weakly dominates q and

denoted as $p \preceq q$. For $\forall i(1 \leq i \leq d)$, $p_i < q_i$, we say p dominates q and denoted as $p \prec q$. If $p \prec q$, we call p dominates q or p is a dominant point of q and q is called a dominated point of p . If q is a dominated point of p and is not a dominated point of any other dominated point of p , then q is called a successor or nearest dominated point of p .

Definition 8 : For a set of matched points $X = \{X^1 = (x_1^1, x_2^1, \dots, x_d^1), \dots, X^m = (x_1^m, x_2^m, \dots, x_d^m)\}$ on a given sequences set $T = \{S_1, S_2, \dots, S_i, \dots, S_d\}$, if X^k is not dominated by any X^j for $j = 1, 2, \dots, m$, Then X^k is called a non-dominated point of set X .

The basic idea of the dominant point based algorithms is to construct a direct acyclic graph (DAG). Their framework is as follows. First, given d sequences, the source point of the graph is defined as a d dimensional point $O(0, 0, \dots, 0)$. There is no input edge for this point and thus its in-degree is 0. The level of the source point is defined as level 0. Next, we find all successor points of the current point O and construct a directed edge from the current point O to its each successor. The level of the successor points is defined as level 1. The set of all dominated points on level-1 is calculated by *non-dominated sorting*(Gustavsson and Syberfeldt (2018)). Then we find all successor points of each non-dominated point on level-1, construct an edge from each non-dominated point on level-1 to its each successor, and mark the level of all these successors as level 2. We find all dominated points on level-2 by *non-dominated sorting*. This process is repeated until no new successor point is created, and then the construction of the DAG is completed. If a point does not have any successor, then its successor is defined as the ending point $\infty(\infty, \infty, \dots, \infty)$. When the DAG is constructed, the character sequence represented by the points in each longest path from the source point to the ending point forms the LCS/MLCS. So the key point of the LCS/MLCS problem is how to construct the DAG.

Example 1 : For sequences $X = ATTTGCGGA$ and $Y = TTATGCCA$, use the dominant points based algorithm to find the MLCS.

1. Initialization.
Create the source point $O(0, 0)$. Its level k is set to $k = 0$. Let D^k denote the set of non-dominated points on level k and V^k denote the set of all successors on level $(k-1)$. Thus $D^0 = \{O(0, 0)\}$ and $V^0 = \emptyset$.
2. Construction of DAG on level-1.
For each point $O(0, 0)$ on D^0 , search its all successors: $A(1, 3)$, $C(5, 6)$, $G(6, 5)$ and $T(2, 1)$, and add a direct edge from point $O(0, 0)$ to its each successor. These successors are all successors on level-0 and all points on level 1. Put them into set V^1 . With non-dominated sorting, the dominated points $C(5, 6)$ and $G(6, 5)$ in set V^1 are removed (note that $A(1, 3) \prec C(5, 6)$, $A(1, 3) \prec G(6, 5)$, $T(2, 1) \prec C(5, 6)$, $T(2, 1) \prec G(6, 5)$, and $A(1, 3)$ and $T(2, 1)$ are non-dominated points. Set $D^1 = \{A(1, 3), T(2, 1)\}$ and $k = k + 1$.
3. Construction DAG on level-2.
For each point in $D^1 = \{A(1, 3), T(2, 1)\}$, search its all successors. For point $A(1, 3) \in D^1$, $A(8, 8)$, $C(5, 6)$, $G(6, 5)$ and $T(2, 4)$ are its all successors and put in V^2 . Add a direct edge from point $A(1, 3)$ to its each successor. For point $T(2, 1)$, $A(8, 8)$, $C(5, 6)$, $G(6, 5)$ and $T(3, 2)$ are its all successors and put in V^2 . Add a direct edge from $T(2, 1)$ to its each successor. Delete the repeated successors in V^2 . With the non-dominated sorting on V^2 , the dominated points $A(8, 8)$, $C(5, 6)$ and $G(6, 5)$ are removed. Let $D^2 = \{T(2, 4), T(3, 2)\}$ and $k = k + 1$.
4. Repeat step 3 until all points in set D^k have no successor. Then put $\infty(\infty, \infty)$ as the successor of points in D^k and the construction of DAG is completed.

The detail process of construction of DAG in example 1 is shown in Figure 1.

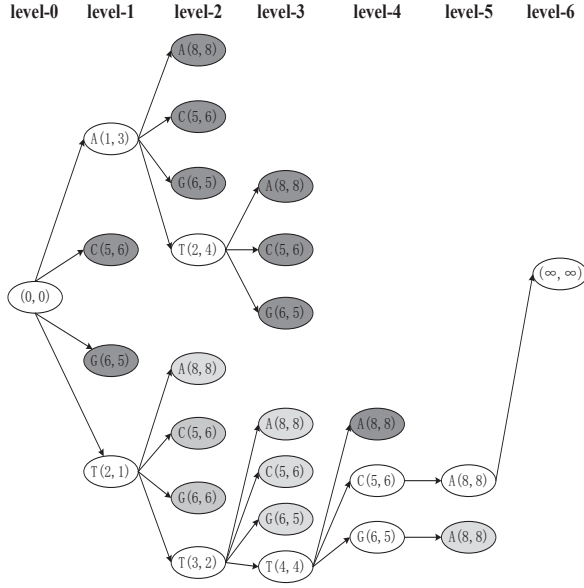


Fig. 1. DAG of sequences X and Y .

From example 1, we can see that the dominant point based algorithms for LCS/MLCS problems have the following disadvantages:

- When a sequence has the consecutively repeated characters, the successors (after the last repeated character) of this repeated character are the same, but for each repeated character, we have to search these successors repeatedly and it is unnecessary. For example, character T appears in X for three consecutive times (i.e., $ATTT\dots$) and in Y for two consecutive times ($TTAT\dots$) in example 1. Each T has the same successors (after these T s). These T s correspond to points $T(2,1)$, $T(3,2)$, $T(2,4)$ and $T(4,4)$, and each has the same successors $A(8,8)$, $C(6,5)$ and $G(5,6)$ (after these points T s) as shown in Figure 1.
- One point may appear multiple times on one level and appears in multiple levels. For example, $A(8,8)$ appears two times on level 2 and appears on levels 2, 3, 4 and 5, respectively in example 1. For these repeated appearance points, only the point with the largest level (not $\infty(\infty, \infty)$) has the actual contribution to LCS/MLCS (e.g., $A(8,8)$ on level 5 in example 1). Thus storing the points which do not connect to the point with the largest level (not $\infty(\infty, \infty)$) wastes a lot of storage (e.g., storing points not connecting to $A(8,8)$ on level 5 in example 1 will waste a lot of storage).
- With the number k of level increasing, the number of points in V^k will increase exponentially when the length and the number of sequences increase, and it will take a lot of time to look for the non-dominated points in set V^k by *non-dominated sorting*.

Note that the time and space complexity of dominant point based algorithm for LCS problems is $O((n+r) \log n)$, where n is the length of two sequences and r is the number of points in DAG. Because r increases exponentially with the number of sequences and the length of sequences, the time and space consumption increase exponentially with the increase of the number of sequences and the sequence length. So for MLCS problems, the time and space complexity is very high. To overcome

these shortcomings, a series of improved dominant point based algorithms were proposed. Korkin(Korkin (2001)) proposed a parallel version of the original dominant point based algorithm with time complexity $O(r|\Sigma|)$, where r is the number of dominant points and $|\Sigma|$ is the size of character set Σ . Chen(Chen *et al.* (2006)) proposed an algorithm called FAST_LCS with new data structure called the successor table to enhance the dominant point based algorithm. The successor table can quickly create subsequent points and remove dominated points in the DAG, and thus Fast_LCS greatly reduces the time required to construct the DAG. Based on FAST_LCS, Wang *et al* proposed an algorithm called Quick-DPPAR (Wang *et al.* (2011)) to improve the process of removing the dominated points in the DAG. Yang (Yang *et al.* (2010)) proposed a parallel MLCS algorithm based on cloud computing platform, but the synchronization cost is too high when solving large-scale MLCS problems. For solving the MLCS problems quickly, Li and Wang proposed a new algorithm called Top_MLCS(Li *et al.* (2016)) which deleted all redundant points (all repeated and all dominated points) by designing an Irredundant Common Subsequence Graph (ICSG). ICSG is much smaller than DAG and thus Top_MLCS is much efficient than the above mentioned algorithms for large-scale MLCS problems. However, when the scale of MLCS problem is extended to a certain extent, the time and space complexity of this algorithm is still too high and Top_MLCS still cannot solve the MLCS problem in an acceptable time and space costs.

In order to solve the large-scale MLAC problems efficiently, especially for sequences with a lot of consecutively repeated characters, we propose a character merging algorithm in Section III and also use an example to help understanding the idea of the proposed algorithm.

3 Character Merging Algorithm (CMA)

As mentioned before, the dominant point based algorithms still have many disadvantages. For example, there are too many redundant points in DAG which results in too large DAG, and the *non-dominated sorting* costs too much time and space, etc. For some specific MLCS problems, e.g., for MLCS problems of DNA sequences which have a lot of consecutively repeated character, it is very possible to make use of this characteristic to design efficient algorithm for the LCS/MLCS problems of DNA sequences. For this purpose, we propose a new algorithm by merging the consecutively repeated characters, which can greatly decrease the length of sequences and thus greatly decrease the time and space consumption, and can solve the large-scale MLCS problems for DNA sequences.

3.1 Framework for Character Merging Algorithm

For the sequences with a lot of consecutively repeated characters (e.g., for DNA sequences), we first merge the consecutively repeated characters in each sequence. The sequences will be shorten. Then we use the shorten sequences (by merging the consecutively repeated character) to construct a weighted DAG (briefly WDAG). Finally, we search the MLCS from the WDAG.

Given a set of d DNA sequences $S = \{S_1, S_2, \dots, S_d\}$, the process of merging consecutively repeated characters is as follows.

Merge consecutively repeated characters

For each DNA sequence S_i in set $|S|$, merge consecutively repeated characters to result in sequence S'_i .

1. Let the first character of sequence S_i be the first character of sequence S'_i .
2. For the second character of sequence S_i :
 - If $S_i[2] \neq S_i[1]$, $S_i[2]$ is added to the end of sequence S'_i , that is, $S'_i[2] = S_i[2]$.

- Otherwise, merge the second character of sequence S_i .
- For the third character of sequence S_i :
 - If $S_i[3] \neq S_i[2]$, then $S_i[3]$ is added to the end of sequence S'_i .
 - Otherwise, merge the third character of the sequence S_i .
 - For the (j) th character of the sequence S_i :
 - If $S_i[j] \neq S_i[j-1]$, then the character $S_i[j]$ is added to the end of sequence S'_i .
 - Otherwise, merge the (j) th character of sequence S_i .
- $j = j + 1$.
- Repeat step 4 until $j > n$.

Through the above steps, we can obtain the merged sequence S'_i of S_i , $|S'_i| \leq |S_i|$. As the length and the number of the DNA sequences increase, the lengths of the shorten sequences will decrease a lot.

Algorithm 1 Merge consecutively repeated characters and mark the number

Input: Sequence set $S = \{S_1, S_2, \dots, S_d\}$, $|S| = d$ and the character set Σ .

Output: Merged sequence set S' and table num

- 1: Create the merged sequences $S' = \{S'_1, S'_2, \dots, S'_d\}$
- 2: Create the table num
- 3: Create now to mark the positions
- 4: **for** $i \leftarrow 1$ to d **do**
- 5: $S'_i[1] \leftarrow S_i[1]$
- 6: $num(i, 1) \leftarrow 1$
- 7: $now \leftarrow 1$;
- 8: **for** $j \leftarrow 2$ to $|S_i|$ **do**
- 9: **if** $S_i[j] == S_i[j-1]$ **then**
- 10: $num(i, now) \leftarrow num(i, now) + 1$
- 11: **else**
- 12: $now \leftarrow now + 1$
- 13: $S'_i[now] \leftarrow S_i[j]$
- 14: $num(i, now) \leftarrow 1$
- 15: **end if**
- 16: **end for**
- 17: **end for**
- 18: Return S' and num

Mark the number of consecutively repeated characters

For each merged sequence S'_i , we need to mark the number of the consecutively repeated characters for each character. The rules for mark are as follows:

1. Construct a table num , where the length of the (i) th row is the length of the (i) th merged sequence S'_i .
2. For S'_i , define row i of num . For each character from the left to the right of S'_i , count the number of consecutively repeated characters in sequence S_i . For example, for the (j) th character from the left of S'_i , if this character consecutively repeats x_{ij} times in S_i , then the element $num(i, j)$ of row i and column j in num will be x_{ij} for $i = 1, 2, \dots, d$ and $j = 1, 2, \dots, |S'_i|$.

We can mark the number of consecutively repeated characters while merge consecutively repeated characters. The detailed steps of merging and marking consecutively repeated characters can be described by the algorithm 1.

Construct the Weighted Directed Acyclic Graph (WDAG) for merged sequences

In order to find MLCS quickly, it is necessary to build a graph similar to DAG as in the existing dominant point based algorithms, but here we will build a graph called Weighted Directed Acyclic Graph (WDAG) for the merged sequences. We need to use a data structure called *Successor Table* to help us to quickly search for the successors of each matched point.

Successor Table (ST): For the sequence set $S = \{S_1, S_2, \dots, S_d\}$, in order to quickly find the successors of all matched points on each sequence S_i , Chen (Chen *et al.* (2006)) proposed the *Successor Table* ST_i for S_i . The *Successor Table* set is denoted as $ST = \{ST_1, ST_2, \dots, ST_d\}$. For a sequence $S_l \in S$ ($|S_l| = n$), its character set is denoted as $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_k\}$. The ST_l for the l th sequence S_l is a matrix which has $|\Sigma|$ rows and $n + 1$ columns (Hunt and Szymanski (1977)). The element $ST_l[i, j]$ in row i and column j is defined as:

$$ST_l[i, j] = \min\{m | S_l[m] = \sigma_i, m > j, 1 \leq i \leq |\Sigma|, 0 \leq j \leq n\}. \quad (3)$$

where $S_l[m]$ represents the character of the m position from left in sequence S_l . From the Formula (3), we can see that $ST_l[i, j]$ mean the minimal position m after the position j in sequence S_l with $S_l[m] = \sigma_i$. It has been proved that the time complexity of finding all successors of a matched point $p = \{p_1, p_2, \dots, p_d\}$ is $O(d|\Sigma|)$ (Chen *et al.* (2006)). The set of all successors of the matched point p can be calculates as (Li *et al.* (2016)):

$$Successor(p) = \{(ST_1[i, p_1], ST_2[i, p_2], \dots, ST_d[i, p_d])\} \quad (4)$$

s.t. $1 \leq i \leq |\Sigma|, \forall ST_l[i, p_l] \neq \emptyset, 1 \leq l \leq d$.

For the merged d sequences, the steps to construct WDAG are as follows and the algorithm for construction of WDAG is shown in Algorithm 2:

Algorithm 2 Algorithm for constructing WDAG of merged sequences

Input: *Successor Table* ST of S' , $ST = \{ST_1, ST_2, \dots, ST_d\}$

Output: WDAG of merged sequence set S'

- 1: source point $0(0, 0, \dots, 0)$, ending point $\infty(\infty, \infty, \dots, \infty)$
- 2: Hash table $h = \{(0, 0, \dots, 0), 0\}$
- 3: Queue $\leftarrow$ source point
- 4: **while** Queue $\neq \emptyset$ **do**
- 5: $p \leftarrow Pop(Queue)$
- 6: $Successor(p) \leftarrow$ all successors of p
- 7: **if** $Successor(p) = \emptyset$ **then**
- 8: Construct Edge $< p, \infty >$
- 9: $Indegree[\infty] \leftarrow Indegree[\infty] + 1$
- 10: **end if**
- 11: **for** $k : Successor(p)$ **do**
- 12: **if** $k \notin Hash\ table$ **then**
- 13: map it to largest mapped number plus 1
- 14: Push(Queue, k)
- 15: **end if**
- 16: Construct Edge $< p, k >$
- 17: $Indegree[k] \leftarrow Indegree[k] + 1$
- 18: **end for**
- 19: **end while**
- 20: Return WDAG

1. Initialization.

Create a queue q to assist in the construction of WDAG. Use a hash

- table to compress storage space and map multidimensional points to a number.
- For the merged d sequences, create a d dimensional point $O(0, 0 \dots 0)$ as the source point of WDAG. There is no input edge and the indegree is 0. Put $O(0, 0 \dots 0)$ into the queue q and map the point $O(0, 0 \dots 0)$ to the number 0 with the hash table.
 - For point $O(0, 0 \dots 0)$, search its all successors in the character set $\Sigma = \{A, C, G, T\} : A(x_1^A, x_2^A \dots x_d^A), C(x_1^C, x_2^C \dots x_d^C), G(x_1^G, x_2^G \dots x_d^G)$ and $T(x_1^T, x_2^T \dots x_d^T)$ (or a part of them). Use the hash table to map the successors of O to the numbers 1, 2, 3, 4, respectively, and put the successors into the queue q .
 - Construct the directed edge from point O to each of its successors, respectively. Remove point O from queue q .
 - From the current queue q , take the first point p and search its all successors in set $\Sigma = \{A, C, G, T\}$. For each successor, determine whether its mapped number exists in the hash table.
 - If it does not exist in hash table, then put it into queue q and map it to the largest number among the mapped numbers plus 1.
 - If it exists in hash table, find its mapped number in the hash table.
 - Add a directed edge from p to its each successor and remove p from queue q .
 - Repeat steps 5 and 6 until queue q is empty.

Edge weight: Each edge weight of DAG in the existing dominant point based algorithms is 1. In WDAG, we define the edge weight as follows:

Definition 8 : For any positions l and r in sequence S'_i ($1 \leq l \leq r \leq |S'_i|$), $W_i[l, r]$ represents the total number of repeated character $S_i[l]$ from the l st position to the r th position in S'_i (excluding the r th position), i.e., It can be calculated by:

$$W_i[l, r] = \sum_j num(i, j) \quad (5)$$

where $j \in \{m | S'_i[m] = S'_i[l], l \leq m < r\}$

Definition 9 : For each edge $E < X(x_1, x_2 \dots x_d), Y(y_1, y_2 \dots y_d) >$ from node $X(x_1, x_2 \dots x_d)$ to node $Y(y_1, y_2 \dots y_d)$ in WDAG of d merged sequences, the edge weight of E is defined as:

$$W[X, Y] = \min_{1 \leq i \leq d} W_i[x_i, y_i] \quad (6)$$

We can maintain the prefix sum of num , so as to avoid repeated accumulation and ensure that $W_i[l, r]$ can be completed in $O(1)$ time complexity. According to the above definition, we can get the edge weight of each edge in the WDAG.

Determine the levels of points in WDAG

Calculate the weight of each edge $E < X(x_1, x_2 \dots x_d), Y(y_1, y_2 \dots y_d) >$ on WDAG by using **formula (6)**. The level of point Y is the sum of the level of point X and the weight of edge $E < X(x_1, x_2 \dots x_d), Y(y_1, y_2 \dots y_d) >$. Then the length of the longest path of WDAG is equal to the maximum level of all points. The specific steps are as follows and the algorithm for determining the levels of points is given in Algorithm 3:

- Initialization.
Create a queue q and put the source point $O(0, 0 \dots 0)$ into the queue. Define the level of the point O as 0. For each point, create a precursor array to record the precursors which are on the longest path from source point to this point.
- For point $O(0, 0 \dots 0)$, define the level of its each successor P by the edge weight of $E < O, P >$.
- For any successor P , Let the indegree of P is decremented by 1. If the indegree of P becomes 0, put P into queue q . Remove point O from queue q .

- From the current queue q , take the first point P and define the level of its successor Q as follows: Case 1. When the level of Q has been defined, the new level of Q is the bigger one between the level of point P plus the edge weight of $E < P, Q >$ and the current level of Q . Case 2. When the level of Q has not been defined, define the level of Q by the level of point P plus the edge weight of $E < P, Q >$.
- The indegree of the successor point Q is decremented by 1. If the indegree of point Q become 0, put point Q into queue q . Remove point P from queue q .
- Repeat steps 4 and 5 until queue q is empty.

Algorithm 3 Determine the levels of points in WDAG

Input: The WDAG of merged sequence set S'

Output: The *level* and *precursor* of each point

```

1: Queue  $\leftarrow \emptyset$ 
2: level[0]  $\leftarrow 0$ 
3: while Queue  $\neq \emptyset$  do
4:    $p \leftarrow Pop(Queue)$ 
5:   for  $k : Successor(p)$  do
6:     if level[p] + W[p, k] > level[k] then
7:       level[k]  $\leftarrow$  level[p] + W[p, k]
8:       precursor(k) =  $\emptyset$ 
9:     end if
10:    if level[p] + W[p, k]  $\geq$  level[k] then
11:      precursor(k)  $\leftarrow p$ 
12:    end if
13:    Indegree[k]  $\leftarrow$  Indegree[k] - 1
14:    if Indegree[k] == 0 then
15:      Push(Queue, k)
16:    end if
17:  end for
18: end while
19: Return level and precursor array

```

Output MLCS

After determining the level of the matched point on WDAG, it can be easily seen that the longest path from the source point to the ending point of WDAG is corresponding to the MLCS and can be obtained by the backward search, i.e., any precursor point of the ending point $\infty(\infty, \infty \dots \infty)$ is on the longest path. Similarly, any precursor point of these points are also on the longest path of WDAG. Therefore, we can traverse WDAG with the depth-first from ending point $\infty(\infty, \infty \dots \infty)$ to the source point, and output the character sequences on the longest path. Then we use the number of the consecutively repeated characters to transform the character sequences into MLCS.

4 Experiments and Analysis

4.1 Data Set and Compared Algorithms

In our experiments, the DNA sequences are chosen from real biological sequences datasets NCBI (Available at: <http://www.ncbi.nlm.nih.gov/nucleotide/110645304?report=fasta>) as the test data. In order to test the performance of the proposed algorithm on multi-sequences and long-sequences, we conduct three types of experiments: 1) Fix the sequence length to 75 and change the number of sequences; 2) Fix the number of sequences to 5 and change the length of sequences; 3) Fix the number of sequences to 3000 and change the length of sequences; The first two types of the experiments are usually adopted by the existing algorithms in their experiments. To demonstrate the performance of the proposed algorithm

CMA, we conduct the third type of the experiments. Also note that in the experiments, the sequences used were chosen randomly in the sequence set and we do not know which sequences were chosen in the existing algorithms. So the existing results in the existing algorithms cannot be directly used to make the comparison. In order to make a fair comparison, we have to conduct the experiments by each compared algorithm using the same data set.

For the first type experiments, we fixed the length of the sequence to 75, and do experiments on 20 instances with the number of sequences increased from 3 to 10000. For the second type experiments, we fix the number of sequences to 5, and conduct the experiments on 16 instances with the length of sequences increased from 60 to 210. For the third type experiments, we fix the number of sequences to 3000, and conduct the experiments on 15 instances with the length of sequences increased from 50 to 120. So we do the experiments on total 51 instances.

To highlight the performance of the proposed algorithm (CMA), we compare CMA with three state-of-the-art algorithms FAST_LCS, Quick-DP, Top_MLCS in the experiments. All of the compared algorithms are run on MacBookPro with Inter Core i5(2.7GHz), 8G memory and 256G Solid State Hard Disk. The operating system is macOS 10.14.1 professional 64. All of the test algorithms are coded and runned in C++ with g++ 4.2.1. The results are shown in Tables 1-3.

Column 1 in Table 1 is the number of DNA sequences, and column 1 in Tables 2 and 3 is the length of DNA sequences, while column 2 in Tables 1-3 is the length of MLCS in the test sequences. In Tables 1-3, columns 3 to 6 are mean running time of FAST_LCS, Quick-DP, Top_MLCS and CMA, respectively, bold face numbers represent the shortest running time on the data set and '-' represents that the results are not obtained when the running time is more than one hour.

For each experiment, we run the algorithms in 5 times, and in each time, the sequences are chosen randomly but they are the same for all compared algorithms. The mean running times in millisecond(ms) are recorded and shown in Tables 1-3.

Table 1. The mean running time (in millisecond) of the algorithms FAST_LCS, Quick-DP, Top_MLCS and CMA on different numbers of DNA sequences with the length of sequences fixed to 75.

Number of sequences	MLCS	DNA ($ \Sigma = 4$)			
		FAST_LCS	Quick-DP	Top_MLCS	CMA
3	38	232	66	94	55
4	34	381	142	558	243
5	31	1420	669	2769	957
6	28	144980	8283	12421	4107
7	27	262907	52047	42648	13175
8	26	436173	177827	103730	29265
9	26	1545742	618911	241073	63798
10	24	—	2665340	342493	90599
60	21	—	—	187371	74514
100	20	—	—	97221	35000
300	18	—	—	28827	15663
500	18	—	—	14875	8048
700	18	—	—	15618	10163
900	14	1539664	792545	2432	1940
1000	14	1585740	848810	2458	2094
2000	13	292471	174863	1688	1421
3000	13	94736	42124	1050	983
4000	12	59640	33959	1205	1176
5000	12	40423	23523	1140	1130
10000	11	34724	21159	1690	1676

Table 2. The mean running time (in millisecond) of the algorithms FAST_LCS, Quick-DP, Top_MLCS and CMA under various lengths of DNA sequences. Each of the data sets contains 5 sequences.

Length of sequences	MLCS	DNA ($ \Sigma = 4$)			
		FAST_LCS	Quick-DP	Top_MLCS	CMA
60	24	247	185	554	293
70	29	996	582	1813	802
80	32	3477	2200	5259	2010
90	39	8946	3290	11958	4455
100	40	16454	10661	24581	7194
110	45	32805	27967	47393	13462
120	50	83451	33811	80704	24739
130	54	155429	87016	158041	44416
140	60	346286	230130	253843	71540
150	64	610453	416808	522508	118254
160	68	1328626	468912	632601	165243
170	72	2329877	1140960	1108910	255914
180	78	—	2471152	2617126	420019
190	80	—	—	—	577337
200	83	—	—	—	990368
210	85	—	—	—	3053120

Table 3. The mean running time (in millisecond) of the algorithms FAST_LCS, Quick-DP, Top_MLCS and CMA under various lengths of DNA sequences. Each of the data sets contains 3000 sequences.

Length of sequences	MLCS	DNA ($ \Sigma = 4$)			
		FAST_LCS	Quick-DP	Top_MLCS	CMA
50	7	220	279	140	198
55	7	353	345	164	219
60	8	1023	711	251	314
65	11	15464	9677	787	1038
70	11	31295	19394	1118	1114
75	12	85768	55941	2143	1883
80	13	310686	199824	3811	2937
85	14	—	—	19979	16691
90	15	—	—	97035	60518
95	17	—	—	152758	101402
100	18	—	—	276477	175263
105	20	—	—	1347435	675237
110	21	—	—	—	1034947
115	21	—	—	—	1677354
120	22	—	—	—	3498774

4.2 Results and Anaysis

It can be seen from Table 1 that when the length of the sequence is fixed to 75, all algorithms can quickly solve the MLCS problems with 3-5 sequences. With the increase of the number of sequences, the mean time of all compared algorithms (FAST_LCS, Quick-DP, Top_MLCS and CMA) will first increase and then decrease. When the number of sequences increase from 6 to 10, the mean time used by all compared algorithms increase, but the proposed CMA used the least time and performed best. When the number of sequences increases larger than 10, the mean time used by all compared algorithms decrease. Especially, when the number of sequences increases from 10 to 700, FAST_LCS cannot find MLCS in the given time, and Quick-DP cannot find the MLCS for most problems. The time used by Top_MLCS is much more than that used by CMA. When the number of sequences increases from 900 to 10000, all compared algorithms can find the MLCS, but CMA performs best.

It also can be seen from the results that the difficulty of the problems does not increase with the increase of the number of sequences. It seems that the most difficult problems for the first type problems are those whose number of sequences is 10-700 (the number of sequences for most difficult problems is in the middle range of 3 to 10000). Among the problems with 10-700 sequences, both FAST_LCS and Quick-DP cannot find the MLCS for all or most of the problems in the given time. However, when the

number of sequences grows to a certain extent (after 900 sequences), all algorithms can solve the MLCS problems in given time and use less time on these problems than that on problems with the number of sequences being 10-700. This is because that for the fixed length of sequences, with the increase of the number of sequences, the cost for searching all MLCS will increase, but when the number of sequences increases to some amount, both the number and the length of MLCS will decrease. This results in the decrease of the cost for searching MLCS.

It can be seen from Table 1 that the mean time of two algorithms FAST_LCS and Quick-DP increases very quickly with the increasing of the number of sequences. The main reason for these two algorithms taking much longer time is that they need too much time to do nondominate sorting (note that when the number of sequences increases, the nondominate sorting needs a lot of time and space costs.), while Top_MLCS and CMA do not need non-dominate sorting and thus use relatively shorter time. It can be also seen that Top_MLCS used much fewer time than FAST_LCS and Quick-DP did, while CMA used much fewer time than Top_MLCS. Note that when the number of DNA sequences reaches 60, the time used by Top_MLCS and CMA began to decrease. This means that even for the most difficult problems for FAST_LCS and Quick-DP, they are not difficult for Top_MLCS and CMA, especially for CMA, because CMA used much fewer time than Top_MLCS (almost half time of Top_MLCS). For large-scale MLCS problems, good performance of CMA is especially obvious. The main reason is that CMA has reduced the scale of the problem by merging the repeated characters, and it uses the extremely short (relatively) time to build and traverse WDAG.

Table 2 shows the results for the second type of experiments, i.e., when the number of the sequence is fixed to 5, the length of DNA sequences increase from 60 to 210. From Table 2, it can be seen that CMA performs best on almost test instances except for three instances with the number of sequences being 60, 70 and 90, respectively, i.e., for instances with relatively small number of sequences. It also can be seen from Table 2 that FAST_LCS runs faster than Top_MLCS when the sequence length is shorter than 120. Also, the running time of Quick-DP is shorter than that of Top_MLCS when the sequence length is shorter than 180. FAST_LCS and Quick-DP outperform Top_MLCS in these cases because the number of DNA sequences is relatively small and the *non-dominated sorting* takes not too much time. However, the running time of CMA grows at the slowest as the length of the DNA sequence increases. When the length of the DNA sequence is greater than 130, the running time of the other three algorithms grows faster than that of CMA. When the sequence length is greater than 170, FAST_LCS algorithm takes a lot of time in nondominate sorting because the number of nodes increases very quickly, which makes it impossible to find MLCS within an acceptable time. When the sequence length is greater than 180, all of FAST_LCS, Quick-DP and Top_MLCS cannot find MLCS at the valid time, and Top_MLCS algorithm cannot handle the problems because it creates too many nodes which exceed the memory limit. Also note that CMA used the much shorter time than three compared algorithms did for most of the problems, and only CMA can find all MLCS on all test data in an acceptable time.

Table 3 shows the results for the third type of experiments. When the length of 3000 sequences is greater than 80, FAST_LCS and Quick_DP can't solve the MLCS problem in given time. As the length of the DNA sequence increases, Top_MLCS algorithm takes longer time to solve the MLCS problem than CMA algorithm. When the length of DNA sequences is greater than 105, only CMA algorithm can solve the large-scale MLCS problem in given time.

It can be seen from the above results that the character merging algorithm (CMA) performs best on large-scale MLCS problems. It can still solve small-scale MLCS problems quickly. Compared with the compared algorithms, the advantages for CMA to solve large-scale MLCS problems

are more obvious. In summary, for large-scale DNA sequences, CMA can find all MLCS and is very effective and efficient.

5 Conclusion

In this paper, we proposes a character merging algorithm (CMA), which merges consecutively repeated characters, shortens the length of sequences, reduces the size of problems to be handled, and thus can solve the large-scale MLCS problem quickly and efficiently. This CMA algorithm is very efficient in solving MLCS with more repeated characters. The proposed algorithm is specially suitable to DNA sequences because DNA sequences have a lot of consecutively repeated characters, and the current existing mainstream algorithms do not make use of this characteristic of the data set. By using the character merging technique, the length of DNA sequences can be reduced by about 25% (Note that NDA sequences have only four characters. The probability of the repeat of the current character and the latter one is also 25%). Thus, the weighted directed acyclic graph (WDAG) proposed in this paper is much smaller than DAG in the existing algorithms, and it needs less storage space and traversal time than DAG does. The experimental results show that the proposed algorithm outperforms three compared state-of-the-art algorithms, especially for large-scale MLCS problems. Thus, Character Merging Algorithm(CMA) is a fast and memory efficient MLCS algorithm for DNA sequences alignment.

Acknowledgements

This work was supported in part by the National Natural Science Foundation of China (61872281) and in part by the Natural Science Foundation of Shaanxi Province under Grant 2016JZ022.

References

- Andrade, H., Nieto, J. J., and Torres, A. (2014). The number of alignments between two dna sequences. *BMC Bioinformatics*, 15,1(2014-04-01), 15(1), 94.
- Aravanis, A. M., Lee, M., and Klausner, R. D. (2017). Next-generation sequencing of circulating tumor dna for early cancer detection. *Cell*, 168(4), 571–574.
- Chaabane, L. (2018). A hybrid solver for protein multiple sequence alignment problem. *Journal of Bioinformatics & Computational Biology*, (2).
- Chen, Y., Wan, A., and Liu, W. (2006). A fast parallel algorithm for finding the longest common sequence of multiple biosequences. *Bmc Bioinformatics*, 7(Suppl 4), 1–12.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms, Third Edition*.
- Guerra, A. A. B. (1992). *Fast linear-space computations of longest common subsequences*. Elsevier Science Publishers Ltd.
- Gustavsson, P. and Syberfeldt, A. (2018). A new algorithm using the non-dominated tree to improve non-dominated sorting. *Evolutionary Computation*, 26(1), 89–116.
- Hirschberg, D. S. (1977). Algorithms for the longest common subsequence problem. *Journal of the Acm*, 24(4), 664–675.
- Hsu, W. J. and Du, M. W. (1984). Computing a longest common subsequence for a set of strings. *Bit Numerical Mathematics*, 24(1), 45–59.
- Hunt, J. W. and Szymanski, T. G. (1977). A fast algorithm for computing longest common subsequences. *Communications of the Acm*, 20(5), 350–353.
- Kim, T. and Shanklin, C. W. (1998). Parallel computation in biological sequence analysis. *IEEE Transactions on Parallel & Distributed Systems*, 9(3), 283–294.
- Korkin, D. (2001). A new dominant point-based parallel algorithm for multiple longest common subsequence problem. *Technical Report TR01-148, Univ. of New Brunswick, Tech. Rep.*
- Li, Y., Wang, Y., Zhang, Z., Wang, Y., Ma, D., and Huang, J. (2016). A novel fast and memory efficient parallel mlcs algorithm for long and large-scale sequences alignments. In *IEEE International Conference on Data Engineering*, pages 1170–1181.
- Maier, D. (1978). *The Complexity of Some Problems on Subsequences and Supersequences*. ACM.

- Masek, W. J. and Paterson, M. S. (1980). A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, **20**(1), 18–31.
- Peng, Z. and Wang, Y. (2017). A novel efficient graph model for the multiple longest common subsequences (mlcs) problem. *Frontiers in Genetics*, **8**, 104.
- Podlipn̑, R. (2009). Understanding bioinformatics. *Biologia Plantarum*, **53**(3).
- Sankoff, D., . (1972). Matching sequences under deletion-insertion constraints. *Proceedings of the National Academy of Sciences of the United States of America*, **69**(1), 4–6.
- Smith, T. F., Waterman, M. S., *et al.* (1981). Identification of common molecular subsequences. *Journal of molecular biology*, **147**(1), 195–197.
- Umar, M., Sarosh, S., and Bassam, Z. (2015). A comparative analysis of multiple sequence alignments for biological data. *Bio-medical materials and engineering*, **26 Suppl 1**(1), S1781.
- Wang, Q., Korkin, D., and Shang, Y. (2011). A fast multiple longest common subsequence (mlcs) algorithm. *IEEE Transactions on Knowledge & Data Engineering*, **23**(3), 321–334.
- Yang, J., Yun, X., and Yi, S. (2010). An efficient parallel algorithm for longest common subsequence problem on gpus. *Lecture Notes in Engineering & Computer Science*, **2183**(1).
- Yang, J., Xu, Y., Sun, G., and Shang, Y. (2013). A new progressive algorithm for a multiple longest common subsequences problem and its efficient parallelization. *IEEE Transactions on Parallel & Distributed Systems*, **24**(5), 862–870.